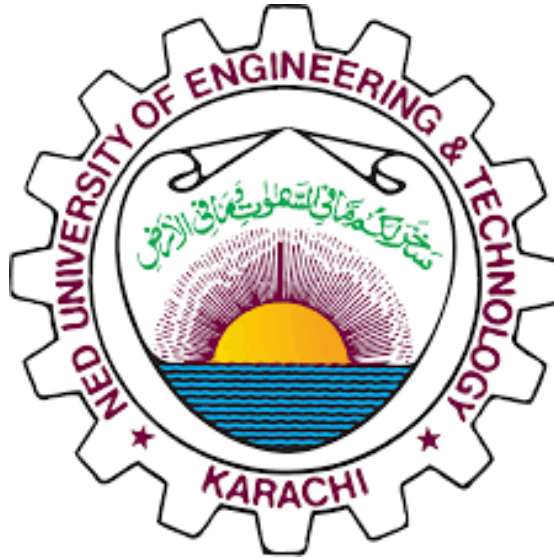




DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

BACHELORS OF SCIENCE IN COMPUTER SCIENCE

Theory of Programming Language **(CT-367)**

GROUP MEMBERS:

LAIBA MINHAJ CT-23001

WANIA MASOOD CT-23002

UNAIZA ASIF CT-23008

FATIMA SAJID CT-23013

Submitted to Sir. Muhammad Kamran

**DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY
BACHELORS OF SCIENCE IN COMPUTER SCIENCE**

Complex Computing Problem Assessment Rubrics

Course Code: CT-367		Course Title: Theory of Programming Language	
Criteria and Scales			
Excellent (3)	Good (2)	Average (1)	Poor (0)
Criterion 1: Understanding the Problem: How well the problem statement is understood by the student			
Understands the problem clearly and clearly identifies the underlying issues and functionalities.	Adequately understands the problem and identifies the underlying issues and functionalities.	Inadequately defines the problem and identifies the underlying issues and functionalities.	Fails to define the problem adequately and does not identify the underlying issues and functionalities.
Criterion 2: Research: The amount of research that is used in solving the problem			
Contains all the information needed for solving the problem	Good research, leading to a successful solution	Mediocre research which may or may not lead to an adequate solution	No apparent research
Criterion 3: Code: How complete the code is along with the assumptions and selected functionalities			
Complete Code according to the according to the selected functionalities of the given case with clear assumptions	Incomplete Code according to the selected functionalities of the given case with clear assumptions	Incomplete Code according to the selected functionalities of the given case with unclear assumptions	Wrong code and naming conventions
Criterion 4: Report: How thorough and well organized is the solution			
All the necessary information clearly organized for easy use in solving the problem	Good information organized well that could lead to a good solution	Mediocre information which may or may not lead to a solution	No report provided

Total Marks: _____

Teacher's Signature: _____

Table of Contents

1. Language Description (C++)	4
2. Selected Functionalities	4
3. Regular Expressions / Rules (Per Functionality)	4
3.1 Lexical Analysis Rules	4
3.2 Keyword Recognition Rule	5
3.3 Semicolon-Free Rule (STMT_END Insertion)	5
3.4 Syntax Grammar Rules	6
3.5 Syntax Error Rules	6
3.6 Semantic Rules	7
4. Source Code (Implementation Overview)	7
Github Link:	7
https://github.com/UnaizaAsif/SafeCPP.git	7
5. Output Description	8
Lexical Phase	8
Syntax Phase	8
Semantic Phase	8
Final Summary	8
6. Limitations	8
7. Assumptions	9
8. Outputs	9
Example Output 1	9
Example Output 2	10
Example Output 3	12

SafeCPP : Static Analyzer

1. Language Description (C++)

C++ is a high-performance, statically typed programming language widely used for system programming, game engines, compilers, and performance-critical applications. It supports procedural, object-oriented, and generic programming paradigms. Due to its low-level memory access capabilities (pointers, manual memory management), C++ is powerful but prone to critical runtime errors such as segmentation faults, memory leaks, and undefined behavior. SafeCPP is designed to detect these issues at compile time using static semantic analysis.

2. Selected Functionalities

SafeCPP implements the following core functionalities:

1. Detection of uninitialized variables (Undefined Behavior)
2. Null pointer dereference detection
3. Memory leak detection
4. Circular include dependency detection
5. Type inference using let
6. Semicolon-optional syntax support
7. Symbol table management
8. Syntax error detection with line/column reporting
9. Lexical tokenization of source code
10. Recognition of keywords, identifiers, literals, operators, and delimiters
11. Comment and whitespace handling

3) Regular Expressions / Rules (Per Functionality)

3.1 Lexical Analysis Rules

- Identifier:

$[A-Za-z][A-Za-z0-9]^*$

- Integer literal:

$[0-9]^+$

- Floating-point literal:

```
[0-9]+\.[0-9]+
```

- String literal:

```
"([^"\\]|\\.)*"
```

- Character:

```
'([^'\\]|\\.).'
```

- Single-line comment:

```
//.*
```

- Multi-line comment:

```
/\*[ \S\S]*?\*/
```

- Operators (multi-character priority):

```
== | != | <= | >= | << | >> | && | || | ->
```

- Operators/delimiters:

```
+ - * / % = ! < > & ( ) { } [ ] ; , . :
```

3.2 Keyword Recognition Rule

If a token matches reserved words such as:

```
int, float, double, char, bool, if, else, while, for, return, let, new,
delete, nullptr, include
```

→ classify as **KEYWORD**, otherwise → **IDENTIFIER**

3.3 Semicolon-Free Rule (STMT_END Insertion)

A virtual statement terminator is inserted when:

- A valid statement-ending token appears (identifier, literal, return, etc.)
- Followed by one or more NEWLINES
- Next meaningful token begins a new statement or block

3.4 Syntax Grammar Rules

- Declaration:

```
<type> <id> [= expression]
```

- Let declaration:

```
• let <id> = expression
```

- Assignment:

```
<id> = expression
```

- If statement:

```
if (expression) statement [else statement]
```

- While loop:

```
while (expression) statement
```

- Function:

```
<type> <id>(params) { body }
```

- Statement termination:

```
; | NEWLINE | STMT_END
```

3.5 Syntax Error Rules

- Missing ;,), {, {, }
- Unexpected tokens in declarations or expressions
- Invalid variable declarations (e.g., int a,,b)
- Empty or incomplete expressions (int a = ;)
- Unexpected operator usage (a == = b)

3.6 Semantic Rules

- **System 1 (Uninitialized usage):**
Detect variables used before assignment
- **System 2 (Null pointer):**
Detect dereference of nullptr or unknown pointer state
- **System 3 (Memory leaks):**
Detect new without delete, including loop-based leaks
- **System 4 (Include analysis):**
Build include graph and detect cycles using DFS
- **System 5 (Let inference):**
 - Integer → int
 - Floating → double
 - String → string
 - Char → char
 - Boolean → bool
 - Else → unknown
- **System 6 (Semicolon-free reporting):**
Reports each virtual statement boundary

4) Source Code (Implementation Overview)

Github Link:

<https://github.com/UnaizaAsif/SafeCPP.git>

- **main.cpp**
Controls execution pipeline (Lexical → Syntax → Semantic)
- **lexer/**
Token generation, classification, and statement termination logic
- **parser/**
Recursive-descent parser with AST construction and error recovery
- **semantic/**
Implements safety systems (1–6), symbol tracking, and type inference

- **utils/**
Symbol table for tracking variable state, initialization, and allocation
- **analyzer/**
Include dependency graph + cycle detection module
- **UI/** (optional)
Express backend + React frontend for visualization

5) Output Description

The compiler produces structured phase-wise output:

Lexical Phase

- Full token stream with type, value, line, column

Syntax Phase

- Syntax errors with precise location
- VALID / INVALID status
- Total error count

Semantic Phase

- System-based warnings:
 - Uninitialized variable usage
 - Null pointer dereference
 - Memory leaks
 - Include cycles
 - Type inference results

Final Summary

- Total safety issues
- Overall program validity status

6) Limitations

- Supports only a limited subset of C++ grammar.
- Uses simplified recursive-descent parsing.
- No complete AST-based semantic analysis.
- Complex expressions are not fully handled.
- Operator precedence handling is limited.

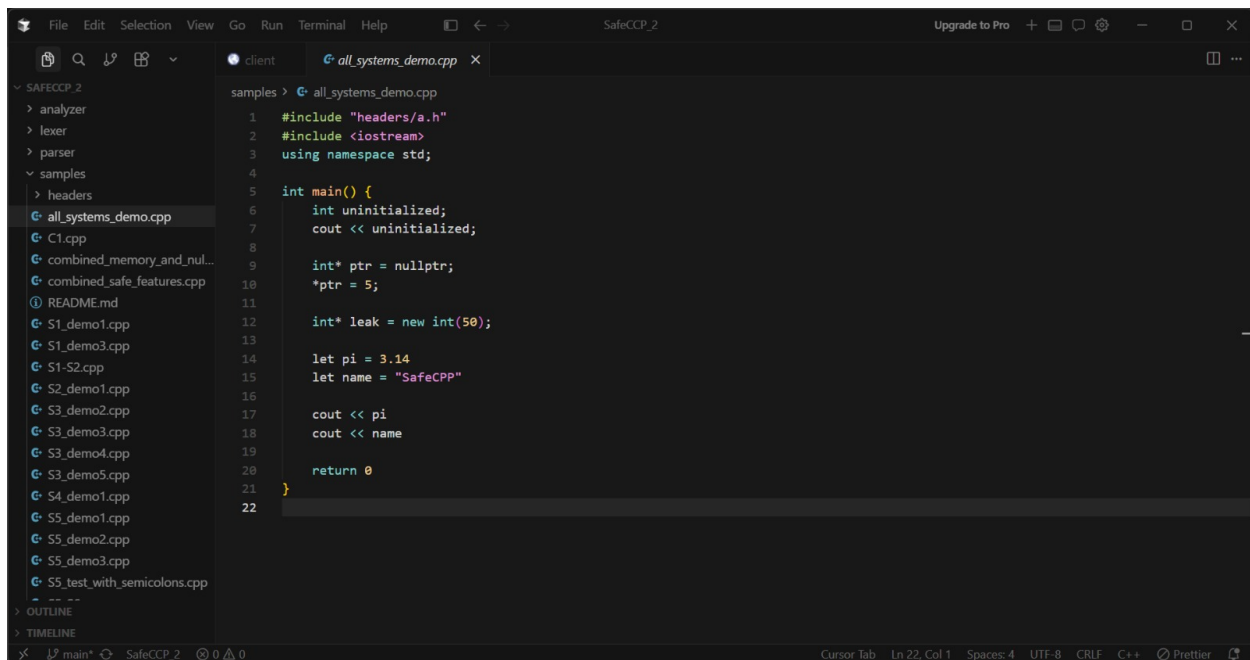
- Loop and control-flow analysis is partial.
- Semantic analysis is mostly token-based.
- Include analysis supports only "file.h" style includes.
- Include checking does not fully verify file existence.
- Syntax recovery may skip deeper errors.
- No runtime checking or code execution is performed.
- No compiler backend or code generation is included.

7) Assumptions

- Input code is written in a simplified C++-like syntax supported by SafeCPP.
- Code is processed in a single pass using tokens (no full execution).
- Variables follow block scope using { } braces.
- Semicolon is optional (newline or STMT_END can end a statement).
- Only new and delete are tracked for memory management.
- Only #include "file.h" style includes are considered for dependency analysis.
- Type inference using let assumes RHS expression is enough to determine type.
- The system performs static analysis only (no runtime execution)

8) Outputs

Example Output 1



The screenshot shows the SafeCPP 2 IDE interface. On the left is a file explorer with a tree view containing folders like 'analyzer', 'lexer', 'parser', 'samples', and 'headers', along with various demo files. The main editor window displays the file 'all_systems_demo.cpp' with the following code:

```

1  #include "headers/a.h"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      int uninitialized;
7      cout << uninitialized;
8
9      int* ptr = nullptr;
10     *ptr = 5;
11
12     int* leak = new int(50);
13
14     let pi = 3.14
15     let name = "SafeCPP"
16
17     cout << pi
18     cout << name
19
20     return 0
21 }
22

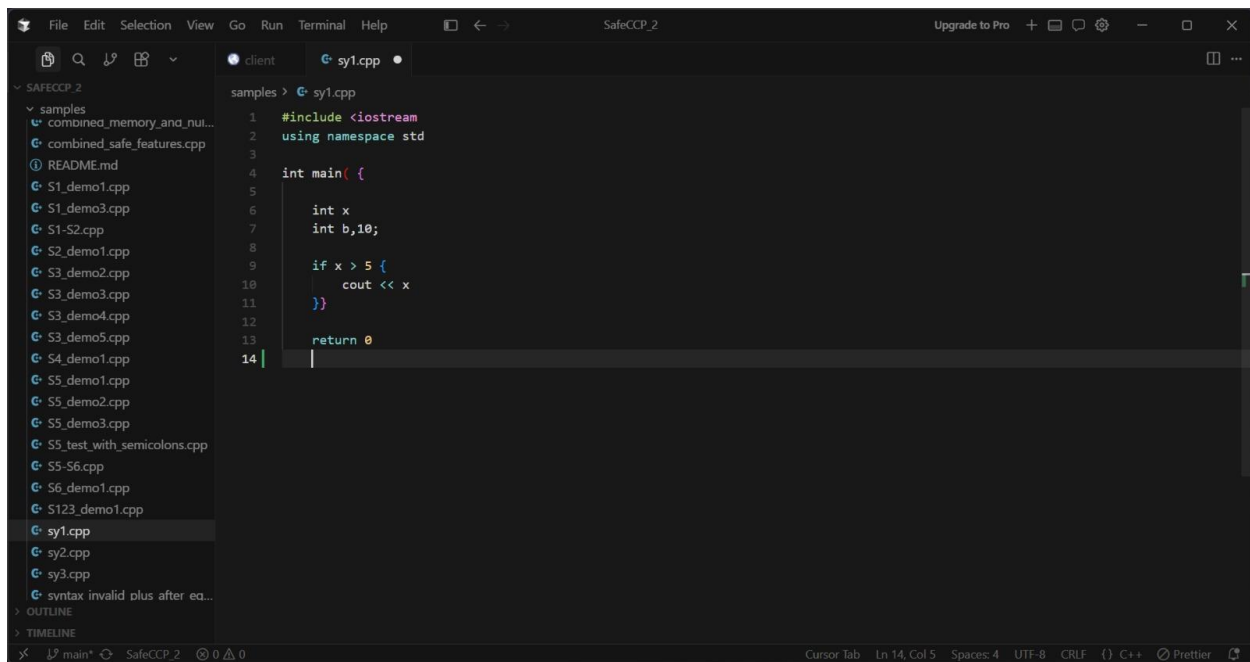
```

The status bar at the bottom indicates 'Cursor Tab', 'Ln 22, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'C++', and 'Prettier'.

The screenshot shows the 'SafeCPP Visualizer' application running in a web browser. The browser tab is labeled 'client' and the address bar shows 'localhost:5175'. The application has a dark theme and a sidebar on the right with a 'Chat' button. The main content area is divided into two panels. The left panel, titled 'LEXICAL TOKENS', displays a 'Token stream' with four tokens: Token-1 (hash symbol), Token-2 ('include'), Token-3 ('headers/a.h'), and Token-4 (newline). Each token is shown with its type, lexeme, and location. The right panel, titled 'SYNTAX ANALYSIS', shows 'Syntax issues found:' with a parse error at line 12, column 17. It also includes a '[SYNTAX SUMMARY]' section with 'SYNTAX STATUS: INVALID' and 'TOTAL SYNTAX ISSUES: 1'. A 'Run Analysis' button is visible in the top right corner of the application interface.

The screenshot shows the 'SafeCPP Visualizer' application with the 'SEMANTIC ANALYSIS' panel on the left and the 'SUMMARY' panel on the right. The 'SEMANTIC ANALYSIS' panel displays several error messages: '[SEMANTIC ERROR] SYSTEM 4: INCLUDE DEPENDENCY ANALYSIS' (circular dependency), '[SEMANTIC ERROR] SYSTEM 1: UNDEFINED BEHAVIOR DETECTION' (uninitialized variable), and '[SEMANTIC ERROR] SYSTEM 2: NULL POINTER SAFETY' (dereferenced pointer). It also includes '[SEMANTIC INFO]' messages for type inference of 'pi' and 'name'. The 'SUMMARY' panel shows 'Total safety issues: 4'. The 'Run Analysis' button is still visible in the top right corner.

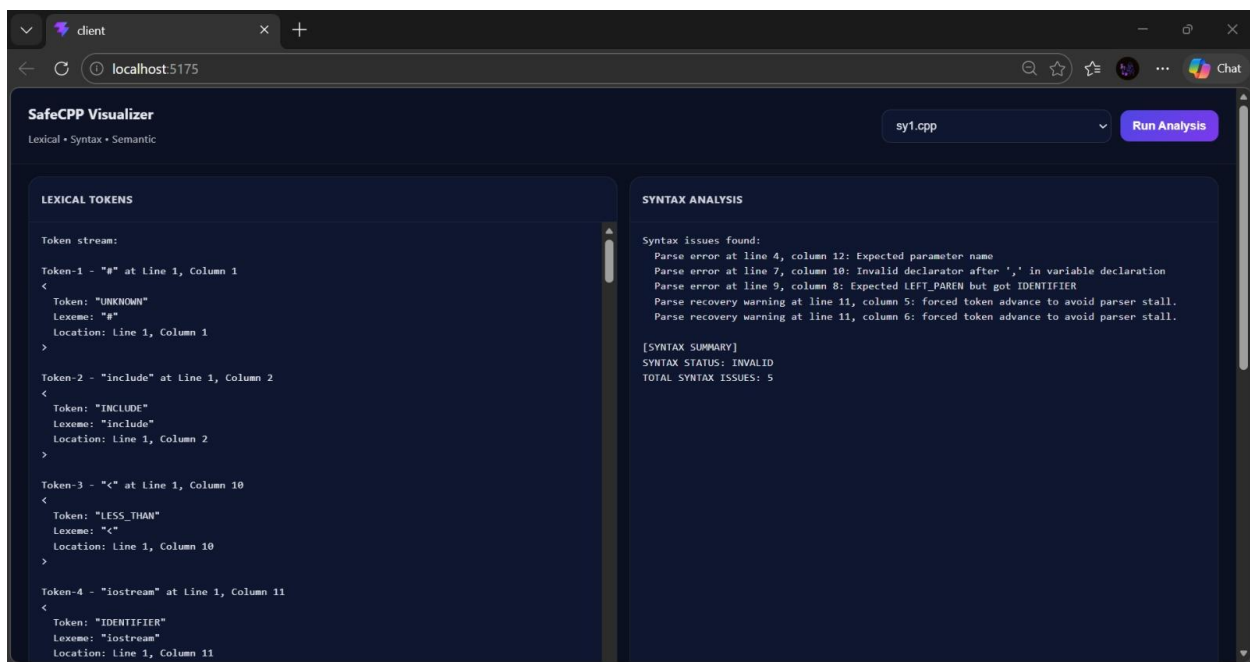
Example Output 2



The screenshot shows a code editor window titled 'SafeCCP_2'. The file 'sy1.cpp' is open, showing the following code:

```
1 #include <iostream>
2 using namespace std;
3
4 int main( {
5
6     int x
7     int b,10;
8
9     if x > 5 {
10         cout << x
11     }
12
13     return 0
14
```

The editor's left sidebar shows a project structure with 'SAFECCP_2' and a 'samples' directory containing various demo files. The status bar at the bottom indicates 'Cursor Tab', 'Ln 14, Col 5', 'Spaces: 4', 'UTF-8', 'CRLF', 'C++', and 'Prettier'.



The screenshot shows the 'SafeCPP Visualizer' web application. The file 'sy1.cpp' is selected, and the 'Run Analysis' button is visible. The application displays the following information:

LEXICAL TOKENS

Token stream:

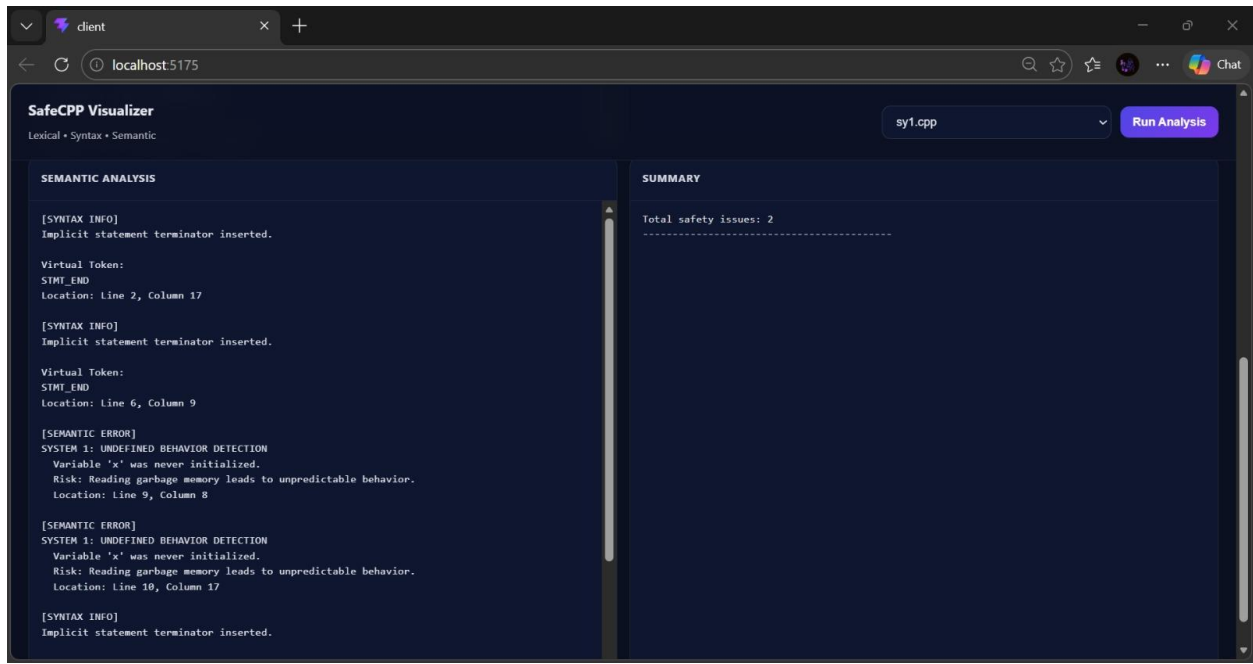
- Token-1 - "#" at Line 1, Column 1
Token: "UNKNOWN"
Lexeme: "#"
Location: Line 1, Column 1
- Token-2 - "include" at Line 1, Column 2
Token: "INCLUDE"
Lexeme: "include"
Location: Line 1, Column 2
- Token-3 - "<" at Line 1, Column 10
Token: "LESS_THAN"
Lexeme: "<"
Location: Line 1, Column 10
- Token-4 - "iostream" at Line 1, Column 11
Token: "IDENTIFIER"
Lexeme: "iostream"
Location: Line 1, Column 11

SYNTAX ANALYSIS

Syntax issues found:

- Parse error at line 4, column 12: Expected parameter name
- Parse error at line 7, column 10: Invalid declarator after ',' in variable declaration
- Parse error at line 9, column 8: Expected LEFT_PAREN but got IDENTIFIER
- Parse recovery warning at line 11, column 5: forced token advance to avoid parser stall.
- Parse recovery warning at line 11, column 6: forced token advance to avoid parser stall.

[SYNTAX SUMMARY]
SYNTAX STATUS: INVALID
TOTAL SYNTAX ISSUES: 5



Example Output 3

